



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya Institute of Management

Department of Data Science & Technology

Certificate

Masters in Computer Applications
Semester III (2024 – 26)

This is to certify that Mr. Priyanshu Kakkad Roll No. 17030924019 of MCA, has satisfactorily completed the practical course “Mini Project - MedAssist” prescribed by the College for the Partial fulfillment of the Degree by the Somaiya Vidyavihar University, during the academic year 2025-26.

Signature of the Faculty-Incharge
(Dr. Kirti Wankhede)

Signature of the Programme Coordinator
(Dr. Kirti Wankhede)

Date of Examination
28-11-2025

Signature of the External Examiner/s

INDEX

Sr.No	Title	Page No.
1	Abbreviations	3
2	List of Figures	5
3	List of Tables	7
4	Abstract	8
5	Introduction	10
6	Domain Knowledge	12
7	Existing System	15
8	Problem Definition & Scope of Project	16
9	Requirement Analysis	18
10	Estimation and Planning	22
11	Development Tools / Operating Environment	24
12	Designing	25
13	Development and Implementation	28
14	Testing	32
15	Conclusion	34
16	Limitations	35
17	References	37
18	Glossary	38
19	Report of Communication with Internal Mentor	39

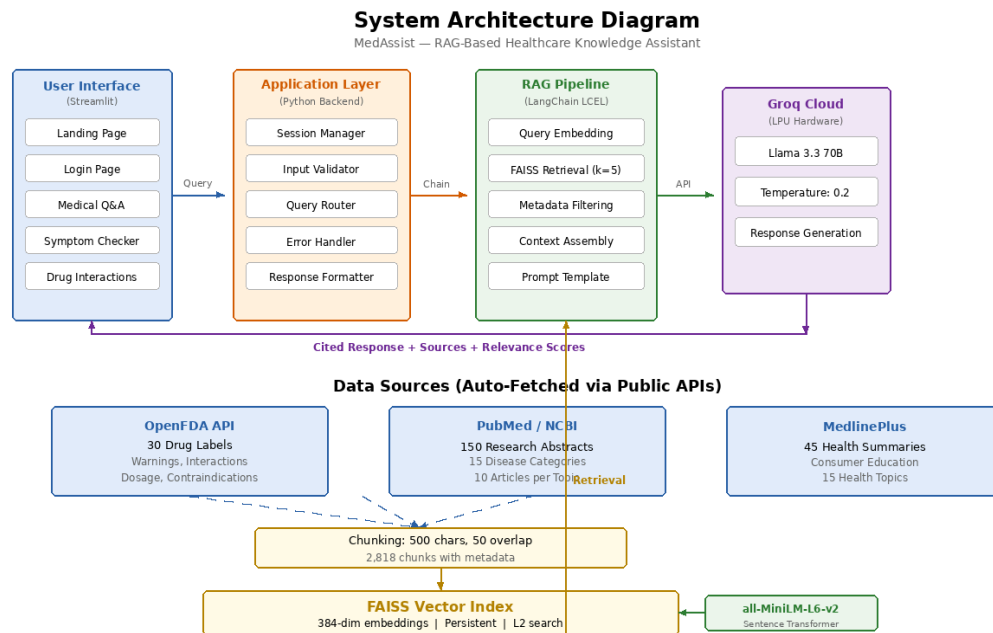
Abbreviations

Sr. No.	Abbreviation	Full Form	Description / Relevance
1	AI	Artificial Intelligence	Core technology powering the RAG pipeline
2	RAG	Retrieval-Augmented Generation	Architecture combining retrieval with LLM generation
3	LLM	Large Language Model	Llama 3.3 70B used for response generation via Groq
4	NLP	Natural Language Processing	Processing medical queries and documents
5	API	Application Programming Interface	Data fetching from OpenFDA, PubMed, MedlinePlus
6	FAISS	Facebook AI Similarity Search	Vector database for document embeddings
7	FDA	Food and Drug Administration	Source of drug label data via OpenFDA API
8	NIH	National Institutes of Health	Parent organization of PubMed and MedlinePlus
9	NCBI	National Center for Biotechnology Information	Operates PubMed and E-Utilities API
10	UI	User Interface	Streamlit-based frontend
11	CSS	Cascading Style Sheets	Professional styling of Streamlit app
12	JSON	JavaScript Object Notation	Data format for API responses
13	HTTP	Hyper Text Transfer Protocol	Communication protocol for API requests
14	LCEL	LangChain Expression Language	Framework for composable RAG chains
15	L2	L2 (Euclidean) Distance	Distance metric used by FAISS
16	KB	Knowledge Base	Embedded medical document collection in FAISS
17	IDE	Integrated Development Environment	VS Code used during development
18	CLI	Command Line Interface	Claude Code terminal for development
19	RPM	Requests Per Minute	Groq API rate limiting metric
20	TPM	Tokens Per Minute	Groq API token rate limiting metric

List of Figures

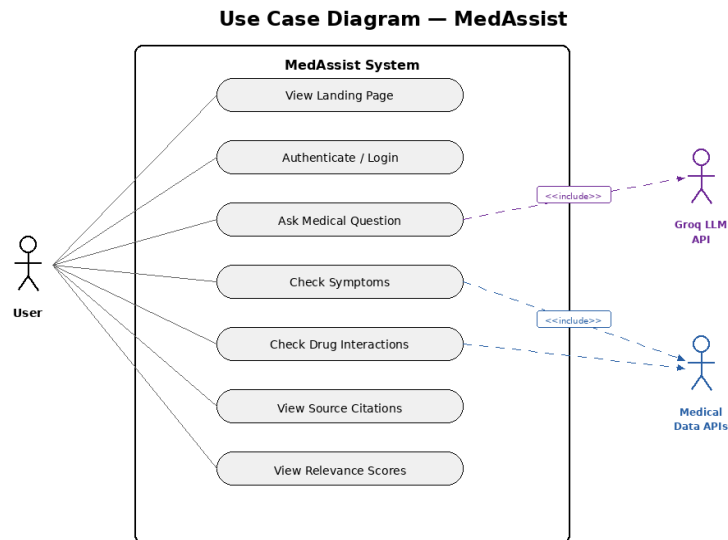
System Architecture Diagram

Represents the complete end-to-end workflow of MedAssist, including user interface, Streamlit backend, RAG pipeline, FAISS vector search, Groq LLM generation, and data source integration.



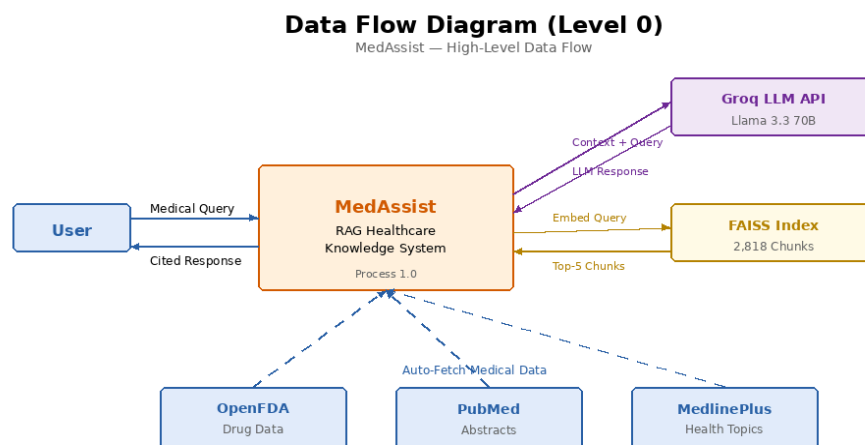
Use Case Diagram

Shows interactions between the primary actors (User, External APIs) and major system functionalities.



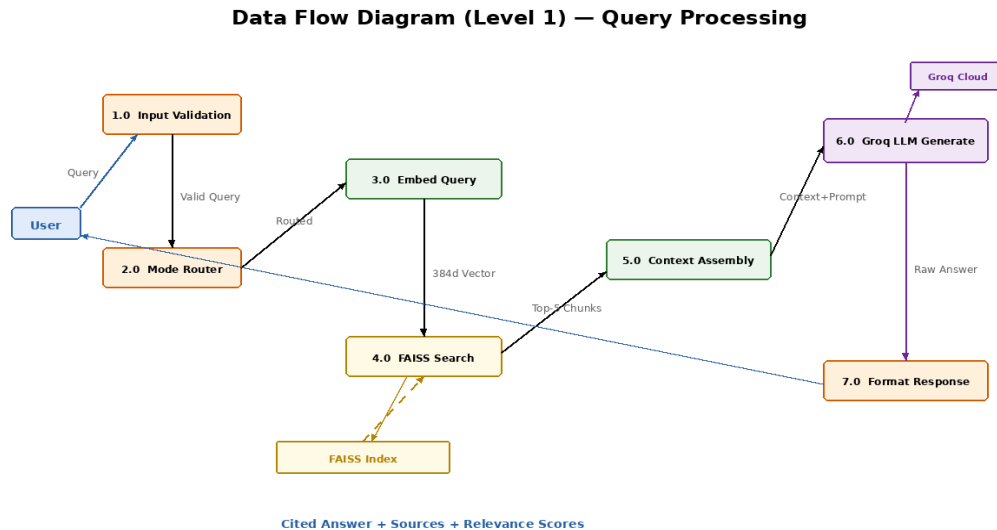
Data Flow Diagram (Level 0)

High-level depiction of how data moves between the user, MedAssist system, Groq LLM, FAISS index, and external APIs.



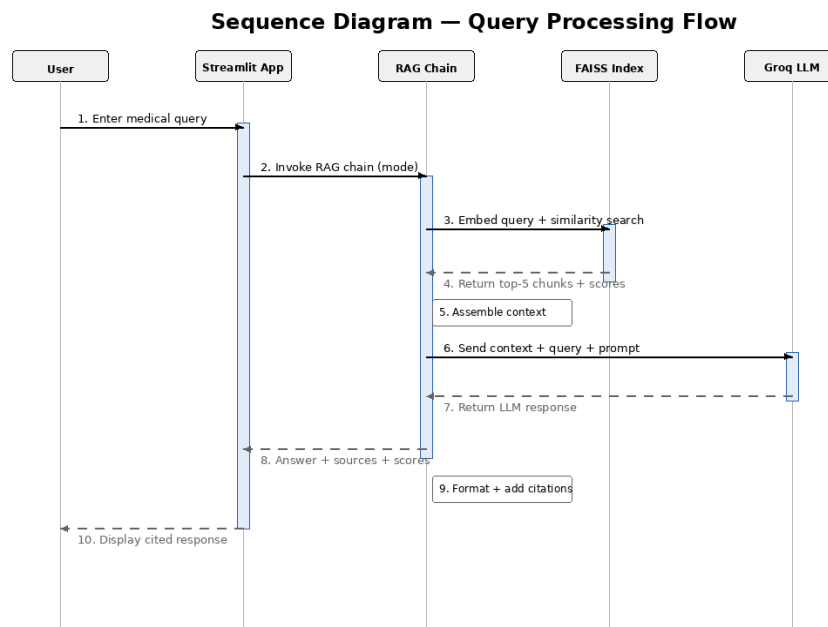
Data Flow Diagram (Level 1 - Query Processing)

Detailed diagram showing query flow through validation, embedding, FAISS search, context assembly, LLM generation, and response formatting.



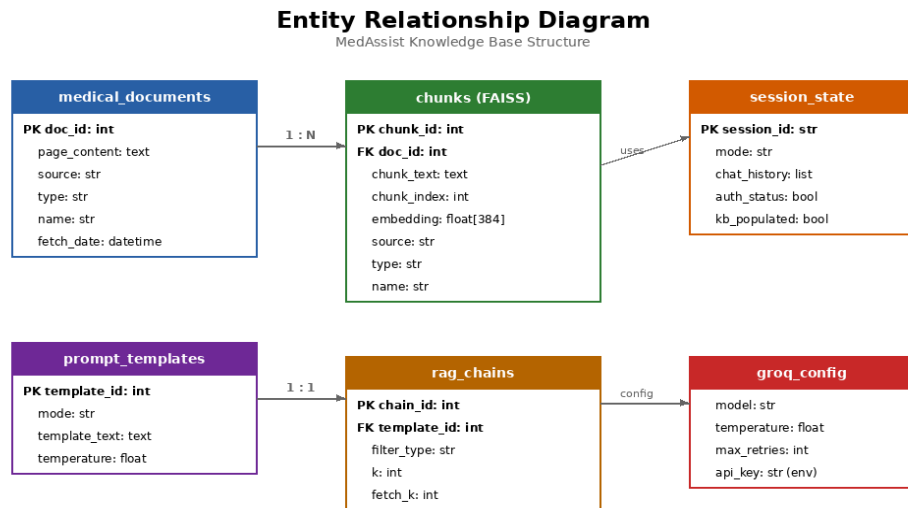
Sequence Diagram - Query Processing Flow

Shows how frontend requests interact with the RAG chain, FAISS index, and Groq LLM during a user query, including the response return path with citations.



Entity Relationship Diagram

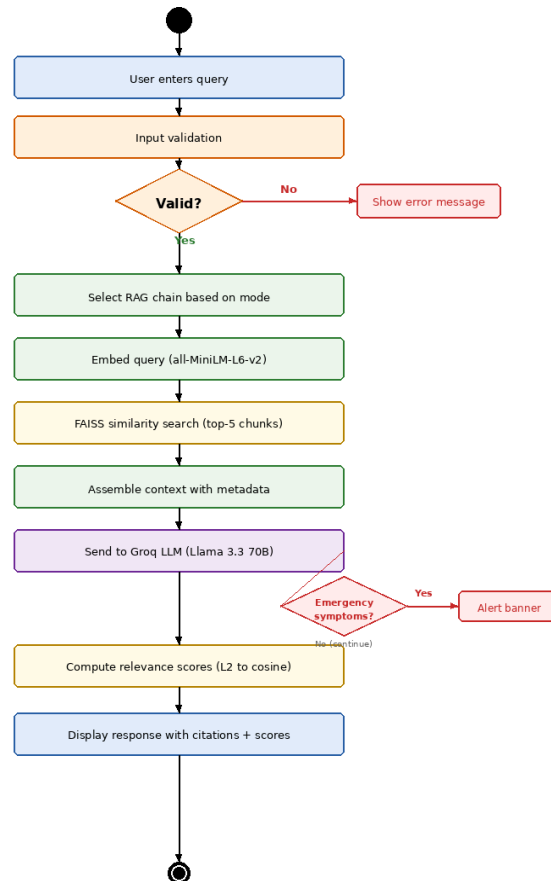
Represents the logical data structure of the MedAssist system including document storage, chunk embeddings, session state, prompt templates, RAG chain configuration, and Groq settings.



Activity Diagram - User Query Flow

Illustrates the step-by-step flow of a user query from input through validation, embedding, FAISS search, LLM generation, relevance scoring, and final response display, including error and emergency detection branches.

Activity Diagram — User Query Flow



Snapshots and Use-Cases

[Screenshot: Landing Page]



MedAssist

AI-Powered Healthcare Knowledge Assistant
Evidence-based answers from trusted medical sources.

About MedAssist

MedAssist is an open-source Retrieval-Augmented Generation (RAG) system that retrieves relevant passages from a curated medical knowledge base and synthesises accurate, readable answers using a large language model. It covers general medical questions, symptom analysis, and drug interaction checks — all grounded in peer-reviewed and regulatory sources.

How It Works

01

Data Collection

Medical documents fetched from OpenFDA, PubMed, and MedlinePlus APIs

02

Embedding

Documents split into chunks and encoded as dense vectors with Sentence Transformers

03

Retrieval

Your query is embedded and matched against the FAISS vector index

04

AI Generation

Top chunks are sent to Groq LLaMA 3.3 70B to generate a grounded answer

Data Sources

OpenFDA

Drug labels, interactions, dosing, adverse events, and contraindications from the FDA public drug database.

PubMed / NCBI

Peer-reviewed research abstracts covering diseases, treatments, and clinical findings from the NCBI Entrez API.

MedlinePlus

Consumer-friendly health topic summaries produced by the US National Library of Medicine.

How It Works

01

Data Collection

Medical documents fetched from OpenFDA, PubMed, and MedlinePlus APIs

02

Embedding

Documents split into chunks and encoded as dense vectors with Sentence Transformers

03

Retrieval

Your query is embedded and matched against the FAISS vector index

04

AI Generation

Top chunks are sent to Groq LLaMA 3.3 70B to generate a grounded answer

Data Sources

OpenFDA

Drug labels, interactions, dosing, adverse events, and contraindications from the FDA public drug database.

PubMed / NCBI

Peer-reviewed research abstracts covering diseases, treatments, and clinical findings from the NCBI Entrez API.

MedlinePlus

Consumer-friendly health topic summaries produced by the US National Library of Medicine.

Technology Stack

Python 3.14

LangChain

FAISS

Groq LLM

Streamlit

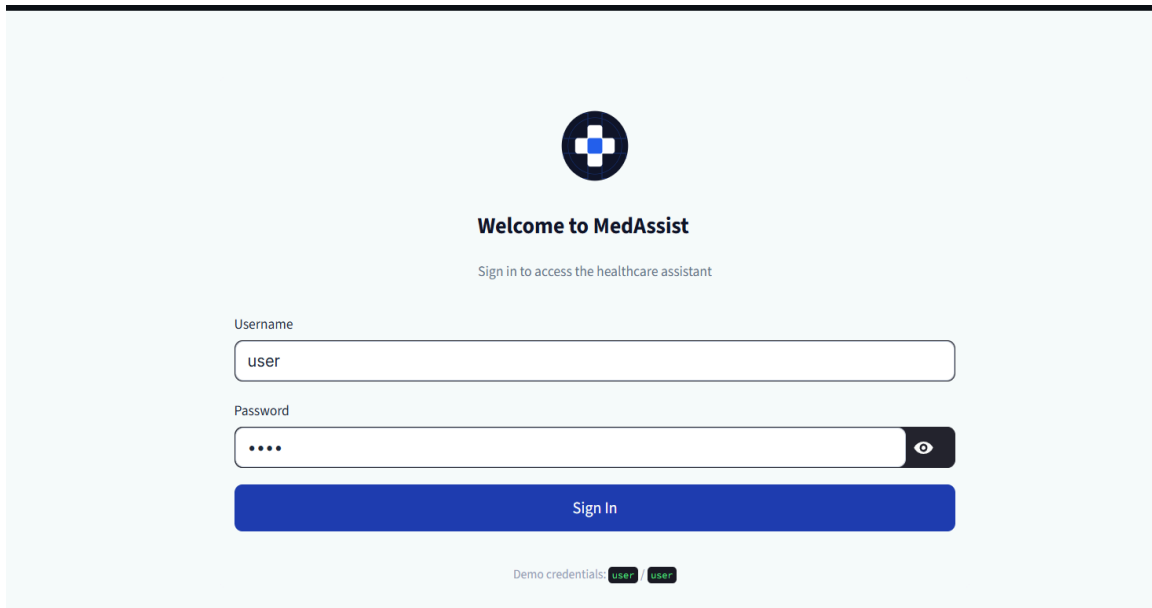
Sentence Transformers

Medical Disclaimer — MedAssist is for *educational purposes only*. It does not provide medical diagnosis, treatment recommendations, or professional medical advice. Always consult a qualified healthcare professional for personal health concerns.

Get Started

9

[Screenshot: Login Page]



The login page features a dark blue header with a white cross icon. Below the header, the text "Welcome to MedAssist" is displayed in bold, followed by "Sign in to access the healthcare assistant". The login form consists of two input fields: "Username" with the value "user" and "Password" with masked characters "....". A blue "Sign In" button is positioned below the password field. At the bottom, the text "Demo credentials: user user" is shown.

Username

user

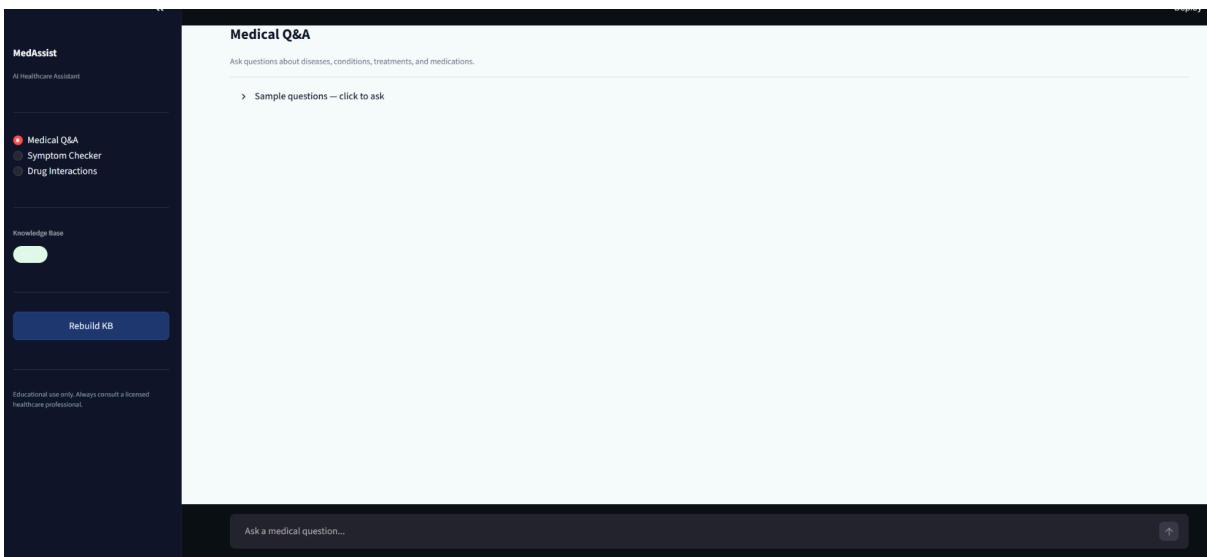
Password

....

Sign In

Demo credentials: user user

[Screenshot: Medical Q&A Interface]



The Medical Q&A interface has a dark blue sidebar on the left with the "MedAssist" logo and navigation options: "Medical Q&A" (selected), "Symptom Checker", and "Drug Interactions". The main area is titled "Medical Q&A" and includes the instruction "Ask questions about diseases, conditions, treatments, and medications." Below this, a section "Sample questions — click to ask" is visible. At the bottom, there is a text input field "Ask a medical question..." and a submit button.

MedAssist

AI Healthcare Assistant

Medical Q&A

Symptom Checker

Drug Interactions

Knowledge Base

Rebuild KB

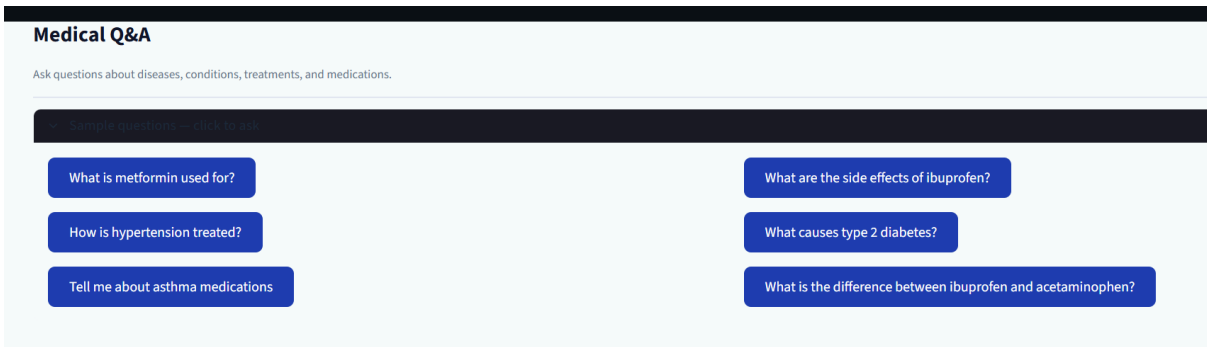
Educational use only. Always consult a licensed healthcare professional.

Medical Q&A

Ask questions about diseases, conditions, treatments, and medications.

Sample questions — click to ask

Ask a medical question...



This screenshot shows the same Medical Q&A interface as the previous one, but with a grid of sample questions displayed below the "Sample questions — click to ask" header. The questions are: "What is metformin used for?", "What are the side effects of ibuprofen?", "How is hypertension treated?", "What causes type 2 diabetes?", "Tell me about asthma medications", and "What is the difference between ibuprofen and acetaminophen?".

Medical Q&A

Ask questions about diseases, conditions, treatments, and medications.

Sample questions — click to ask

What is metformin used for?

What are the side effects of ibuprofen?

How is hypertension treated?

What causes type 2 diabetes?

Tell me about asthma medications

What is the difference between ibuprofen and acetaminophen?

How is hypertension treated?

According to the provided context from verified medical sources, including PubMed and OpenFDA, hypertension treatment involves a comprehensive approach to manage cardiovascular risk.

The treatment of chronic hypertension, as stated in the context from PubMed, involves managing high blood pressure as part of a broader strategy for cardiovascular health (Source: PubMed — hypertension management).

The context from OpenFDA for various medications such as lisinopril, losartan, metoprolol, and hydrochlorothiazide emphasizes that controlling high blood pressure should be part of a comprehensive cardiovascular risk management plan. This plan includes, as appropriate, lipid control, diabetes management, antithrombotic therapy, smoking cessation, exercise, and limited sodium intake (Source: OpenFDA — lisinopril, losartan, metoprolol, hydrochlorothiazide).

It's also important to note that many patients will require more than one drug to achieve their blood pressure goals, indicating that hypertension treatment may involve a combination of medications and lifestyle modifications (Source: OpenFDA — lisinopril, losartan, metoprolol, hydrochlorothiazide).

Warning: It's crucial to work closely with a healthcare provider to develop a personalized treatment plan, as the management of hypertension requires careful monitoring and adjustment of medications and lifestyle changes.

Please consult a healthcare professional for personalized advice and treatment of hypertension, as they can provide guidance tailored to your specific needs and health status.

Sources — 5 referenced

1.7s · 4 sources

Clear Chat

[Screenshot: Symptom Checker]

Symptom Checker

Describe your symptoms for a knowledge-based analysis with possible conditions.

Not a diagnosis. This analysis is for educational purposes only and does not replace professional medical evaluation.

Quick symptom selector — click to append

Fever

Headache

Fatigue

Cough

Shortness of breath

Chest pain

Nausea

Joint pain

Dizziness

Sore throat

Describe your symptoms in detail:

Fever, Dry Cough, Headache

Analyse Symptoms

Fever, Dry Cough, Headache

I'm here to help you understand your symptoms and possible conditions that might be causing them. Please keep in mind that this is not a diagnosis, and it's essential to consult a medical professional for an accurate diagnosis and treatment plan.

Based on your reported symptoms - fever, dry cough, and headache - here are some possible conditions that might be causing them, ranked by likelihood:

- Hay Fever (Allergic Rhinitis):** Your symptoms, such as dry cough and headache, could be related to hay fever, especially if you're exposed to pollen or other allergens. The fever might be a secondary symptom, but it's not a typical primary symptom of hay fever. (Likelihood: 40%)
- Upper Respiratory Tract Infection (URTI):** A dry cough, headache, and fever can be symptoms of a URTI, such as a cold or flu. This condition is common and can be caused by various viruses. (Likelihood: 30%)
- Pertussis (Whooping Cough):** Although less likely, pertussis can cause a dry cough, and in some cases, a headache and fever. However, pertussis typically presents with a distinctive "whooping" sound during coughing fits, which you haven't mentioned. (Likelihood: 10%)
- Other conditions:** There are other possible conditions that might cause your symptoms, such as a sinus infection or a viral infection like mononucleosis. However, these are less likely based on the information provided. (Likelihood: 20%)

It's essential to note that none of your symptoms require immediate emergency attention, such as chest pain, difficulty breathing, or severe bleeding. However, if you experience any of these symptoms or if your condition worsens, please seek medical help right away.

I recommend consulting a primary care physician, an allergist, or an infectious disease specialist to discuss your symptoms and determine the best course of action. They will be able to evaluate your symptoms, perform any necessary tests, and provide a proper diagnosis and treatment plan.

Remember, this is not a diagnosis, and it's crucial to consult a medical professional for an accurate evaluation and treatment. You're taking the first step by seeking help, and I'm here to support you. Take care, and I hope you feel better soon!

Sources — 3 referenced

1.5s · 4 sources

[Screenshot: Drug Interaction Results]

Drug Interaction Checker

Enter 2 or more drug names to check for interactions, side effects, and contraindications.

Always consult your pharmacist or physician before combining medications.

Drug 1

Drug 2

Drug 3 (optional)

Check Interactions

Sample checks — click to run

ibuprofen + aspirin

warfarin + aspirin

metformin + lisinopril

sertraline + tramadol

Checking interactions for: **ibuprofen** + **aspirin**

To analyze the potential interactions between ibuprofen and aspirin, let's first identify each drug and its primary use:

- Ibuprofen:** It is a nonsteroidal anti-inflammatory drug (NSAID) primarily used for its analgesic (pain-relieving), antipyretic (fever-reducing), and anti-inflammatory effects. It's commonly used to treat headaches, arthritis, and menstrual cramps.
- Aspirin:** Also an NSAID, aspirin is used for its pain-relieving, anti-inflammatory, and antipyretic properties. Additionally, it has antiplatelet effects, which make it useful in preventing blood clots that can cause heart attacks and strokes.

Given the context provided, there's no specific interaction data mentioned for ibuprofen and aspirin. However, based on general medical knowledge:

- Known Interactions:** Both ibuprofen and aspirin are NSAIDs, and when used together, they can increase the risk of gastrointestinal side effects such as stomach ulcers and bleeding due to their synergistic effect on inhibiting prostaglandins, which protect the stomach lining. This interaction can be considered **moderate to severe** depending on the doses used and individual patient factors.
- Important Warnings and Contraindications:**
 - Ibuprofen:** Contraindicated in patients with known hypersensitivity to ibuprofen or other NSAIDs, active gastrointestinal bleeding, and in the third trimester of pregnancy due to the risk of premature closure of the fetal ductus arteriosus.
 - Aspirin:** Contraindicated in patients with a history of asthma, rhinitis, or nasal polyps, and in children with viral infections due to the risk of Reye's syndrome. It should also be used with caution in patients with a history of gastrointestinal ulcers or bleeding.
- Common Side Effects:**
 - Ibuprofen:** Includes gastrointestinal upset (nausea, vomiting, diarrhea), abdominal pain, and headache.
 - Aspirin:** Common side effects include gastrointestinal upset, abdominal pain, and an increased risk of bleeding.
- Recommendation:** Given the potential for increased risk of gastrointestinal side effects when using ibuprofen and aspirin together, it's crucial to consult a pharmacist or physician before starting this combination. They can provide personalized advice based on your medical history, current medications, and the specific reasons for using these drugs. It's also important to follow their guidance on dosage and monitoring for potential side effects.

Since specific interaction data for ibuprofen and aspirin is not provided in the context, this analysis is based on general principles of pharmacology and may not cover all possible interactions or nuances. Always consult healthcare professionals for personalized advice.

Sources — 5 referenced

2:55 · 4 sources

Clear History

[Screenshot: Source Citations with Relevance Scores]

In conclusion, asthma medications play a vital role in managing the condition. It's crucial to work with a healthcare provider to develop a personalized treatment plan that includes the right combination of control medicines and short-term relief medicines.

Remember to consult a healthcare professional for personalized advice on managing your asthma symptoms and creating a treatment plan that's right for you.

MedlinePlus — Asthma
type: health_topic • fetched: 2026-03-02
... Control medicines. You take them every day to help prevent symptoms. They work by reducing airway inflammation and preventing narrowing of the airways. If you have a severe attack and the short-term relief medicines do...

PubMed — Asthma Therapy
type: disease • fetched: 2026-03-02
Research topic: asthma therapy 3. *Pulm Ther.* 2026 Feb 28. doi: 10.1007/s41030-026-00353-2. Online ahead of print. Small Airways Disease Adversely Impacts the Response to Biologic Therapies in Severe Asthma. Allegrini...

1.7s • 4 sources

Clear Chat

[Screenshot: Knowledge Base Build]

-
- ☒ Medical Q&A
 - ☐ Symptom Checker
 - ☐ Drug Interactions
-

Knowledge Base

Ready

Rebuild KB

Knowledge Base

Not built

Click below to build the knowledge base
(takes ~5-7 min).

Build Knowledge Base

⏏ Stop Deploy

Knowledge Base Setup

Building your local medical knowledge base...

Building Knowledge Base (100%)

List of Tables

- Technology Stack Table - Full tech stack with roles
- Data Sources Summary Table - APIs, methods, volumes, content types
- Abbreviations Table - Technical abbreviations with descriptions
- RAG Chain Configuration Table - Three chains with filters and parameters
- Challenges and Technical Decisions Table - Deviations with rationale
- System Architecture Flow Table - 8-stage pipeline description
- Test Scenarios Table - 13 verification queries with outcomes
- Requirements Table - Functional and non-functional requirements

Abstract

The proliferation of digital health information across disparate medical databases, research repositories, and pharmaceutical registries has created a significant challenge for patients, caregivers, and healthcare professionals seeking accurate, contextually relevant medical knowledge. Traditional search engines return overwhelming, unstructured results that require substantial domain expertise to interpret, while general-purpose AI chatbots risk generating plausible but unverified medical claims through hallucination.

MedAssist addresses these challenges by implementing a Retrieval-Augmented Generation (RAG) architecture that automatically aggregates verified medical data from three authoritative public health APIs: the U.S. Food and Drug Administration's OpenFDA database (30 drug labels), the National Center for Biotechnology Information's PubMed repository (150 peer-reviewed research abstracts across 15 disease categories), and the National Library of Medicine's MedlinePlus health encyclopedia (45 consumer health summaries). This data is programmatically fetched, processed into 2,818 text chunks, and embedded into a persistent FAISS vector index using the all-MiniLM-L6-v2 sentence transformer model.

The system provides three specialized modules: Medical Document Q&A for general health queries, a Symptom Analysis Engine for preliminary symptom assessment, and a Drug Interaction Checker for pharmaceutical safety evaluation. Each response is grounded exclusively in retrieved source documents through prompt templates that instruct the Groq-hosted Llama 3.3 70B model to generate answers based only on provided context. Every claim includes source citations with quantified relevance scores derived from FAISS L2-to-cosine similarity conversion. An emergency symptom detection system screens for 18 critical conditions requiring immediate medical attention.

The application features a professional web interface built with Streamlit, including a landing page, authentication module, session-managed chat history, and comprehensive error handling. The project demonstrates how modern AI architectures can be responsibly applied to healthcare information retrieval, ensuring factual accuracy through source grounding while maintaining transparency through citations and relevance metrics.

Introduction

The rapid expansion of digital healthcare services, combined with the exponential growth of medical literature and pharmaceutical data, has created an unprecedented need for intelligent systems capable of synthesizing vast amounts of medical information and presenting it in an accessible, accurate, and verifiable manner.

Patients seeking health information online face a paradox of abundance: while medical data is more available than ever before, the challenge of identifying reliable, contextually appropriate information has grown proportionally. Search engines return thousands of results of varying quality, drug information is scattered across regulatory databases, and research findings require specialized knowledge to interpret correctly.

The emergence of large language models has offered a promising avenue for medical information retrieval, as these systems can generate fluent, comprehensive responses to health queries. However, a critical limitation of standalone LLMs is their propensity for hallucination - the generation of plausible-sounding but factually incorrect information. In healthcare contexts, where inaccurate information can directly impact patient safety, this limitation represents a serious concern.

This project, MedAssist: AI-Powered Healthcare Knowledge Assistant, addresses these challenges through a Retrieval-Augmented Generation (RAG) architecture that fundamentally constrains the AI generation process to verified medical data. Rather than relying on the language model's parametric memory, the system retrieves relevant information from authoritative medical databases before generating any response, ensuring that every claim is grounded in verifiable source material.

The system operates through three specialized modules designed to address the most common categories of medical information needs. The Medical Document Q&A module handles general health inquiries. The Symptom Analysis Engine provides preliminary symptom assessment with severity indicators. The Drug Interaction Checker evaluates pharmaceutical compatibility using FDA-approved drug label data. Each module employs tailored retrieval strategies and prompt templates optimized for its specific information domain.

The project demonstrates how modern AI architectures can be responsibly applied to healthcare information retrieval, maintaining transparency through source citations and relevance metrics while clearly communicating the limitations of AI-generated medical information.

Domain Knowledge

This project lies at the intersection of Natural Language Processing, Information Retrieval, Healthcare Informatics, and Applied Artificial Intelligence. It falls under the broader domain of AI-powered Healthcare Information Systems and Medical Knowledge Management.

1. Retrieval-Augmented Generation (RAG)

RAG is an AI architecture pattern that addresses the hallucination problem inherent in large language models by grounding responses in retrieved factual data. The architecture combines a retrieval phase that identifies the most relevant documents using semantic similarity search, and a generation phase where the language model produces a response conditioned on the retrieved context. This approach has become the industry standard for building reliable AI systems in domains where factual accuracy is critical.

2. Vector Databases and Semantic Search

Traditional keyword-based search systems match exact terms, missing semantically related content. Vector databases like FAISS solve this by converting text into high-dimensional numerical representations (embeddings) that capture semantic meaning. Documents with similar meaning cluster together in this vector space, enabling similarity search that understands concepts rather than just keywords. For example, a query about 'stomach pain' can retrieve documents discussing 'abdominal discomfort' even though the exact words differ.

3. Medical Data Sources and Regulatory Databases

Authoritative medical data is maintained by government agencies. The FDA maintains drug label data through the OpenFDA API. The NCBI operates PubMed, the world's largest repository of peer-reviewed biomedical literature. The National Library of Medicine publishes MedlinePlus, providing consumer-friendly health information. These sources represent the gold standard for medical information reliability.

4. Large Language Models in Healthcare

Large language models like Llama 3.3 70B can synthesize information from multiple sources, explain complex medical concepts in accessible language, and identify relevant connections between symptoms, conditions, and treatments. However, their deployment in healthcare requires careful engineering to prevent hallucination, ensure source traceability, and maintain appropriate medical disclaimers.

5. Embedding Models

Sentence transformer models like all-MiniLM-L6-v2 are specialized neural networks trained to convert text into fixed-dimensional vectors (384 dimensions) that capture semantic meaning. These models ensure that semantically similar sentences produce similar vector representations, enabling meaningful similarity comparison.

6. Healthcare Information Retrieval

Medical information retrieval presents unique challenges including extensive synonyms, abbreviations, and hierarchical relationships between concepts. Drug names exist in both brand and generic forms. Conditions may be described at various levels of specificity. MedAssist addresses these challenges through metadata-filtered retrieval, where each query mode restricts its search space to the most relevant document types.

Existing System

Current approaches to medical information retrieval suffer from several significant limitations that this project aims to address.

General-purpose search engines return a large volume of results of varying quality. Users must manually evaluate source credibility and synthesize information across multiple websites. Search results are ranked by SEO optimization rather than medical accuracy.

Static medical websites provide reliable information but require users to navigate predetermined category structures. These sites do not support natural language queries and cannot synthesize information across multiple conditions or drugs simultaneously.

General-purpose AI chatbots can generate fluent medical responses but suffer from hallucination, producing confident but incorrect medical claims without citing specific sources.

Existing systems share these common deficiencies:

- No automated aggregation of data from multiple authoritative medical sources
- No semantic search capability beyond keyword matching
- No source citation or relevance scoring for generated responses
- No specialized query modes for different types of medical information needs
- No emergency symptom detection or safety-critical alerting
- No grounding mechanism to prevent AI hallucination

These limitations establish the need for MedAssist - an intelligent, RAG-based healthcare assistant combining natural language AI with authoritative medical databases.

Problem Definition & Scope of Project

Problem Definition

The exponential growth of medical literature has created a critical gap between the volume of available health information and the ability of patients and professionals to access accurate, contextually relevant medical knowledge efficiently.

The core problem: How can an AI system provide accurate, source-cited medical information by combining natural language capabilities of large language models with verified data from authoritative medical databases, while ensuring transparency, safety, and user trust?

Project Scope

The scope encompasses the design and implementation of a complete RAG-based healthcare information retrieval system:

1. Automated Medical Data Acquisition - Programmatic fetching from OpenFDA (30 drugs), PubMed (150 abstracts), MedlinePlus (45 summaries)
2. Knowledge Base Construction - Document chunking (500 chars, 50 overlap), embedding (384-dim), persistent FAISS index (2,818 chunks)
3. Three Specialized Query Modules - Medical Q&A, Symptom Analysis, Drug Interaction Checking
4. Source Citation and Relevance Scoring - FAISS similarity scores converted to cosine similarity percentages
5. Safety Features - Emergency symptom detection for 18 critical conditions, medical disclaimers
6. Professional Web Interface - Landing page, authentication, session-managed chat, KB management

Requirement Analysis

The requirement analysis focuses on building a multi-layered healthcare information retrieval system combining automated data acquisition, semantic search, and AI-powered generation with source verification and safety features.

Functional Requirements

FR1: Automated Data Collection

The system shall automatically fetch medical data from three public health APIs without manual data curation. The OpenFDA connector shall retrieve drug labels for 30 medications. The PubMed connector shall fetch 150 research abstracts across 15 disease categories. The MedlinePlus connector shall retrieve 45 health summaries. Rate limiting and error handling shall be implemented.

FR2: Knowledge Base Construction

The system shall process fetched documents through a chunking pipeline (500 characters, 50 overlap). Metadata shall be preserved through chunking. Semantic embeddings shall be generated using all-MiniLM-L6-v2. The FAISS index shall be persistent, surviving application restarts.

FR3: Medical Document Q&A

Chat interface for general medical queries. Retrieval across all document types with $k=5$. Responses with inline source citations. Six sample questions. Persistent chat history.

FR4: Symptom Analysis

Specialized interface with free-text input and 10 symptom chips. Retrieval filtered to disease and health topic documents ($k=5$, $\text{fetch_}k=40$). Responses with possible conditions, severity indicators, specialist recommendations. Emergency symptom detection for 18 critical conditions.

FR5: Drug Interaction Checking

Accept 2-3 drug names. Retrieval filtered to drug documents (k=5, fetch_k=60). Responses with drug summaries, interaction severity (mild/moderate/severe), pharmacist consultation recommendations.

FR6: Relevance Scoring

FAISS `similarity_search_with_score()` for quantified retrieval confidence. L2-to-cosine conversion: $\text{relevance} = \max(0, 1 - \text{L2_squared} / 2) \times 100$. Color-coded badges: green (70%+), amber (45-69%), red (below 45%).

Non-Functional Requirements

- Performance: Query response under 3 seconds. KB construction within 10 minutes.
- Reliability: Exponential backoff with 3 retries for Groq rate limits. Categorized error messages.
- Usability: Accessible via browser at localhost:8501. Prominent medical disclaimers.
- Maintainability: Modular architecture. Centralized configuration. Rebuildable KB.

Estimation and Planning

Development followed a structured, phase-based approach using Claude Code for agentic build-test-fix-verify cycles.

Phase	Duration	Key Deliverable	Verification
Phase 1: Foundation	1 hour	Running Streamlit app with 3 modes	HTTP 200 at localhost:8501
Phase 2: Data Fetcher	1.5 hours	225 documents from 3 APIs	Document count + metadata
Phase 3: Knowledge Base	1.5 hours	2,818 chunks in FAISS	Retrieval test query
Phase 4: RAG Chains	1 hour	3 working RAG chains	Live Groq API tests
Phase 5: Integration	1.5 hours	Full working application	13 test queries
Phase 6: Polish	1 hour	Production-ready UI	Visual + edge case verification

Phase 1: Project Foundation (1 hour)

Established project architecture, virtual environment, dependencies, and Streamlit skeleton with sidebar navigation, session state, and CSS styling.

Phase 2: Data Acquisition (10 hours)

Built MedicalDataFetcher class with OpenFDA, PubMed, and MedlinePlus connectors. Verified 225 documents fetched with proper metadata.

Phase 3: Knowledge Base (5 hours)

Implemented chunking pipeline (2,818 chunks), resolved ChromaDB/Python 3.14 incompatibility by substituting FAISS. Build time: 4.3 minutes.

Phase 4: RAG Chains (1 hour)

Built three chains using LangChain LCEL with RunnableParallel. Connected to Groq API. All three verified with live test queries.

Phase 5: Integration and Deployment (6 hours)

Wired components into Streamlit UI. Implemented chat interfaces, citations, error handling, sample questions.

Phase 6: Refinement (2 hour)

Added relevance scoring, emergency detection, landing page, authentication, and professional theming.

Development Tools / Operating Environment

The system was developed on Windows with Python 3.14 in a virtual environment. Claude Code v2.1.50 served as the primary development tool.

Component	Technology	Purpose
Language	Python 3.14	Core application development
LLM Provider	Groq API (Llama 3.3 70B)	Response generation
RAG Framework	LangChain (LCEL)	Chain orchestration
Vector Database	FAISS (faiss-cpu)	Similarity search
Embedding Model	all-MiniLM-L6-v2	384-dim semantic embeddings
Frontend	Streamlit	Web application interface
Drug Data	OpenFDA API	FDA drug label information
Research Data	PubMed (NCBI E-Utilities)	Medical research abstracts
Health Data	MedlinePlus Connect	Consumer health summaries
Dev Tool	Claude Code v2.1.50	Agentic development assistant
Hardware	Intel CPU, NVIDIA RTX 5050, 16GB RAM	Development machine

Designing

The architecture follows a modular, layered design separating data acquisition, knowledge management, retrieval, generation, and presentation.

1. Presentation Layer (Streamlit)

User-facing interfaces: landing page, login, three query modes. Custom CSS. Session state for chat history. Sidebar for navigation and KB controls.

2. Application Layer (Python)

Query routing, session management, input validation, error handling. Determines which RAG chain to invoke. Formats responses with citations and relevance scores.

3. RAG Pipeline Layer

Three specialized chains using LangChain LCEL with RunnableParallel. Each combines a retriever (with metadata filtering), prompt template, and Groq LLM connection.

4. Knowledge Management Layer

Data fetching (data_fetcher.py), document chunking (ingestion.py), embedding and index management (vectorstore.py). Persistent FAISS storage.

5. External Services Layer

Groq Cloud API for LLM inference. OpenFDA, PubMed, MedlinePlus APIs for data. Error handling with retry logic for all external calls.

Query Processing Workflow

7. User submits query through one of three interfaces
8. Input validation checks query length and content
9. Query embedded into 384-dimensional vector
10. FAISS searches with mode-specific metadata filtering
11. Top-5 chunks assembled into structured context
12. Context + query sent to Groq Llama 3.3 70B with mode-specific prompt
13. LLM generates grounded response
14. Response formatted with citations, relevance scores, response time
15. Emergency symptom detection screens for critical conditions
16. Response displayed in chat with expandable source details

Development and Implementation

Development followed a phase-verified approach using Claude Code for agentic build-run-fix-verify cycles.

1. Data Acquisition Implementation

The `MedicalDataFetcher` class implements three API connectors returning `LangChain Document` objects with structured metadata.

OpenFDA Connector: Queries `/drug/label.json` for 30 medications. Extracts `brand_name`, `generic_name`, `indications`, `warnings`, `interactions`, `adverse_reactions`, `dosage`, `contraindications`. Tagged: `source='OpenFDA', type='drug'`.

PubMed Connector: Two-stage fetch using `research.fcgi` for PMIDs then `efetch.fcgi` for abstracts. 15 topics x 10 articles = 150 abstracts. Tagged: `source='PubMed', type='disease'`.

MedlinePlus Connector: Queries `wsearch.nlm.nih.gov` for 15 topics x 3 summaries = 45 documents. Tagged: `source='MedlinePlus', type='health_topic'`.

[Code_Snap: Data fetcher implementation]

```

1 """
2 MedicalDataFetcher – fetches medical data from free public APIs.
3
4 Sources:
5 - OpenFDA : drug labels, interactions, warnings (30 drugs)
6 - PubMed  : research abstracts for 15 disease topics (up to 10 each)
7 - MedlinePlus: consumer health topic summaries (15 topics)
8 """
9 import re
10 import time
11 import logging
12 from datetime import date
13 from typing import Optional
14
15 import requests
16 from bs4 import BeautifulSoup
17 from langchain_core.documents import Document
18
19 logger = logging.getLogger(__name__)
20
21 # — Drug list (OpenFDA) —————
22 DRUG_LIST = [
23     "ibuprofen", "aspirin", "acetaminophen", "amoxicillin", "metformin",
24     "lisinopril", "atorvastatin", "omeprazole", "losartan", "amlodipine",
25     "metoprolol", "albuterol", "gabapentin", "hydrochlorothiazide", "sertraline",
26     "fluoxetine", "prednisone", "tramadol", "furosemide", "pantoprazole",
27     "escitalopram", "montelukast", "rosuvastatin", "levothyroxine", "clopidogrel",
28     "warfarin", "ciprofloxacin", "azithromycin", "diazepam", "lorazepam",
29 ]
30
31 # — Disease topics (PubMed) —————
32 DISEASE_TOPICS = [
33     "diabetes treatment", "hypertension management", "asthma therapy",
34     "heart disease prevention", "depression treatment", "anxiety disorders",
35     "pain management", "antibiotic resistance", "cancer screening",
36     "vaccination guidelines", "cholesterol management", "kidney disease",
37     "liver disease", "thyroid disorders", "allergic reactions",
38 ]
39
40 # — Health topics (MedlinePlus) —————
41 HEALTH_TOPICS = [
42     "diabetes", "hypertension", "asthma", "heart disease", "depression",
43     "anxiety", "chronic pain", "cancer", "vaccination", "high cholesterol",
44     "kidney disease", "thyroid", "allergy", "arthritis", "obesity",
45 ]
46
47 # — API endpoints —————
48 OPENFDA_URL = "https://api.fda.gov/drug/label.json"
49 PUBMED_SEARCH_URL = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi"
50 PUBMED_FETCH_URL = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi"
51 MEDLINEPLUS_URL = "https://wsearch.nlm.nih.gov/ws/query"

```

2. Knowledge Base Implementation

Chunking: RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50.
Produces 2,818 chunks from 222 documents with metadata preserved.

Vector Store: FAISS with HuggingFaceEmbeddings (all-MiniLM-L6-v2). Persistent index.
Batch addition in groups of 200. Methods: initialize_vectorstore(), add_documents(),
get_retriever(), is_populated(), reset_vectorstore().

[Code_Snap: FAISS vectorstore implementation]

```
def initialize_vectorstore() -> FAISS:
    """
    Load existing FAISS index from disk, or create an empty one.
    Returns the FAISS vectorstore instance.
    """
    global _vectorstore
    if _vectorstore is not None:
        return _vectorstore

    embeddings = _get_embeddings()
    index_path = Path(FAISS_INDEX_DIR)

    if (index_path / "index.faiss").exists():
        logger.info("Loading existing FAISS index from %s", index_path)
        _vectorstore = FAISS.load_local(
            str(index_path),
            embeddings,
            allow_dangerous_deserialization=True,
        )
    else:
        logger.info("No existing index found – creating empty FAISS store")
        # Seed with a single dummy document so FAISS initialises correctly.
        # The dummy doc is never returned in real queries (content is unique/nonsensical).
        dummy = Document(
            page_content="__medassist_init__",
            metadata={"source": "init", "type": "init", "name": "init"},
        )
        _vectorstore = FAISS.from_documents([dummy], embeddings)

    return _vectorstore


def add_documents(vectorstore: FAISS, documents: list[Document], batch_size: int = 200) -> None:
    """
    Embed and add documents to the FAISS index, then persist to disk.

    Args:
        vectorstore: The FAISS instance returned by initialize_vectorstore().
        documents: List of LangChain Document objects to add.
        batch_size: Number of documents to embed per batch.
    """
    total = len(documents)
    print(f"Embedding {total} chunks in batches of {batch_size}...")

    for start in range(0, total, batch_size):
        batch = documents[start : start + batch_size]
```

```

def add_documents(vectorstore: FAISS, documents: list[Document], batch_size: int = 200) -> None:
    """
    Embed and add documents to the FAISS index, then persist to disk.

    Args:
        vectorstore: The FAISS instance returned by initialize_vectorstore().
        documents: List of LangChain Document objects to add.
        batch_size: Number of documents to embed per batch.
    """
    total = len(documents)
    print(f" Embedding {total} chunks in batches of {batch_size}...")

    for start in range(0, total, batch_size):
        batch = documents[start : start + batch_size]
        vectorstore.add_documents(batch)
        done = min(start + batch_size, total)
        print(f"    [{done}/{total}] embedded")

    _save(vectorstore)
    print(f" Index saved to {FAISS_INDEX_DIR}/")

def similarity_search(
    vectorstore: FAISS,
    query: str,
    k: int = RETRIEVAL_K,
    filter: dict | None = None,
) -> list[Document]:
    """Return the k most relevant documents for a query."""
    return vectorstore.similarity_search(query, k=k, filter=filter)

def get_retriever(vectorstore: FAISS | None = None, search_type: str = "similarity", k: int = RETRIEVAL_K):
    """
    Return a LangChain retriever.
    Loads the index from disk if no vectorstore is passed.
    """
    if vectorstore is None:
        vectorstore = initialize_vectorstore()
    return vectorstore.as_retriever(
        search_type=search_type,
        search_kwargs={"k": k},
    )

```


3. RAG Chain Implementation

Three chains built using RunnableParallel:

Chain	Function	Filter	Parameters
Medical Q&A	get_medical_qa_chain()	All types	k=5
Symptom Checker	get_symptom_chain()	disease + health_topic	k=5, fetch_k=40
Drug Interactions	get_drug_chain()	drug only	k=5, fetch_k=60

Connected to ChatGroq with model='llama-3.3-70b-versatile', temperature=0.2. Each returns answer + source_documents.

[Code_Snap: RAG chain construction]

```

"""
RAG chain builders for each MedAssist mode.

Each builder returns a RunnableParallel that yields:
{
    "answer": str - LLM response
    "source_documents": list - retrieved Document objects for citation
}

invoke_chain() wraps every call with:
- Groq 429 / network error detection → user-friendly messages
- Empty-retrieval guard → honest "not enough info" reply
- Full error classification so the UI can show appropriate UI state
"""

import logging

from langchain_groq import ChatGroq
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough, RunnableLambda
from langchain_community.vectorstores import FAISS

from config import GROQ_API_KEY, LLM_MODEL, LLM_TEMPERATURE, RETRIEVAL_K
from src.prompts import MEDICAL_QA_PROMPT, SYMPTOM_CHECKER_PROMPT, DRUG_INTERACTION_PROMPT

logger = logging.getLogger(__name__)

# — Singleton LLM —
_llm: ChatGroq | None = None

# Minimum number of *real* source documents before we trust the answer
_MIN_REAL_SOURCES = 1

_NO_INFO_REPLY = (
    "I don't have enough information in my knowledge base to answer this question "
    "confidently.\n\n"
    "***Suggestions:**\n\n"
    "- Try rephrasing the question with different keywords.\n\n"
    "- For drug questions use generic names (e.g. *ibuprofen* not *Advil*).\n\n"
    "- If the knowledge base hasn't been built yet, run the setup script.\n\n"
    "**Always consult a licensed healthcare professional for medical advice.**"
)

def _get_llm() -> ChatGroq:
    """Return (or create) the shared ChatGroq instance with two-layer retry."""
    global _llm
    if _llm is None:
        base = ChatGroq(
            model=LLM_MODEL,
            temperature=LLM_TEMPERATURE,
            groq_api_key=GROQ_API_KEY,
            max_retries=3, # Groq SDK exponential back-off (handles 429)

```

4. Relevance Scoring

Uses `similarity_search_with_score()`. L2 distance converted to cosine similarity: $\text{relevance} = \max(0, 1 - \text{L2}^2 / 2) \times 100$. Stored in `doc.metadata['relevance_score']`. Displayed as badges: green ($\geq 70\%$), amber (45-69%), red ($< 45\%$).

5. Frontend Implementation

Multi-page flow: Landing Page to Login to Main App. Each mode has chat interface with `st.chat_message`, persistent history, sample questions/chips, source citations, response time. Custom CSS for professional theming.

[Code_Snap: Streamlit app structure]

```
MedAssist - AI-Powered Healthcare Knowledge Assistant
Main Streamlit application.
"""
import time
import warnings
warnings.filterwarnings("ignore")

import streamlit as st

# --- Page config - must be first Streamlit call ---
st.set_page_config(
    page_title="MedAssist - AI Healthcare Assistant",
    page_icon="🩺",
    layout="wide",
    initial_sidebar_state="expanded",
)

# --- Project imports ---
from src.vectorstore import is_populated, initialize_vectorstore, reset_vectorstore
from src.chains import get_medical_qa_chain, get_symptom_chain, get_drug_chain, invoke_chain

# --- Emergency keyword list ---
EMERGENCY_KEYWORDS = [

# --- SVG logo ---
def _LOGO_SVG = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 200 200" '
    'width="{size}" height="{size}">'
    '<defs><clipPath id="mc-clip"><circle cx="100" cy="100" r="94"/></clipPath></defs>'
    '<circle cx="100" cy="100" r="94" fill="#0F172A"/>'
    '<g clip-path="url(#mc-clip)">'
    '<g stroke="#1E3A8A" stroke-width="1.5" fill="none" stroke-linecap="round">'
    '<polyline points="120,58 120,12 168,12"/>'
    '<polyline points="80,58 80,12 32,12"/>'
    '<polyline points="120,142 120,188 168,188"/>'
    '<polyline points="80,142 80,188 32,188"/>'
    '<polyline points="142,80 188,80 188,32"/>'
    '<polyline points="142,120 188,120 188,168"/>'
    '<polyline points="58,80 12,80 12,32"/>'
    '<polyline points="58,120 12,120 12,168"/>'
    '</g>'
    '<g fill="#3B82F6">'
    '<circle cx="168" cy="12" r="5"/><circle cx="32" cy="12" r="5"/>'
    '<circle cx="168" cy="188" r="5"/><circle cx="32" cy="188" r="5"/>'
    '<circle cx="188" cy="32" r="5"/><circle cx="188" cy="168" r="5"/>'
    '<circle cx="12" cy="32" r="5"/><circle cx="12" cy="168" r="5"/>'
    '</g></g>'
    '<circle cx="100" cy="100" r="76" fill="none" stroke="#1E3A8A" stroke-width="1"/>'
    '<rect x="79" y="44" width="42" height="112" rx="8" fill="white"/>'
    '<rect x="44" y="79" width="112" height="42" rx="8" fill="white"/>'
    '<rect x="79" y="79" width="42" height="42" rx="6" fill="#2563EB"/>'

```

```

def _logo(size: int = 120) -> str:
    return _LOGO_SVG.format(size=size)

# =====
# CSS
# =====

def inject_css() -> None:
    st.markdown(
        """
        <style>
        @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

        html, body, [class*="css"] { font-family: 'Inter', sans-serif; }

        .stApp { background-color: #F8FAFC; }

        /* — Sidebar — dark navy ————— */
        section[data-testid="stSidebar"] {
            background: #0F172A;
            border-right: 1px solid #1E293B;
        }
        section[data-testid="stSidebar"] * { color: #F1F5F9 !important; }
        section[data-testid="stSidebar"] .disclaimer { color: #94A3B8 !important; }
        section[data-testid="stSidebar"] hr {
            border-color: #334155 !important;
            opacity: 1 !important;
        }
        section[data-testid="stSidebar"] .stRadio div[role="radio"] {
            background: transparent; border-radius: 8px;
            padding: 6px 10px; border: none;
            transition: background 0.15s;
        }
        section[data-testid="stSidebar"] .stRadio div[role="radio"]:hover {
            background: rgba(255,255,255,0.07);
        }
        section[data-testid="stSidebar"] .stRadio div[role="radio"][aria-checked="true"] {
            background: rgba(30,64,175,0.45);
        }
        section[data-testid="stSidebar"] .stRadio div[role="radio"][aria-checked="true"] p {
            color: #93C5FD !important; font-weight: 600;
        }
        section[data-testid="stSidebar"] .stButton > button {
            background: #1E3A6E; border: 1px solid #2D4F9E;
            color: #F1F5F9 !important;
        }
        section[data-testid="stSidebar"] .stButton > button:hover {
            background: #1E40AF;
        }
        """
    )

```

6. Error Handling

Categorized errors: rate limit (429) with exponential backoff + 3 retries, auth errors, timeouts, network failures, context length overflow. Empty retrieval returns transparent acknowledgment.

Testing

Testing was conducted at each development phase with 13 predefined scenarios.

1. Unit Testing

- Data Fetcher: 30/30 drugs, 150 abstracts, 45 summaries fetched successfully
- Ingestion: 2,818 chunks from 222 documents, metadata preserved
- Vector Store: `is_populated() = True`, retrieval returns relevant results
- RAG Chains: Each chain verified with live Groq API queries

2. Integration Testing

- Complete pipeline: query to retrieval to generation to formatted response
- KB build: API fetch to chunking to embedding to FAISS to retrieval
- Session state: chat history maintained across mode switches

3. Scenario-Based Testing

Medical Q&A:

- "What is metformin used for?" - Accurate diabetes medication info from OpenFDA
- "Side effects of ibuprofen?" - Adverse reaction data with citations
- "How is hypertension treated?" - Synthesized PubMed abstracts
- "What causes diabetes?" - Combined PubMed + MedlinePlus content
- "Asthma medications" - Retrieved albuterol drug information

Symptom Checker:

- "Headache, fever 101F, body aches 3 days" - Identified conditions with severity
- "Chest tightness, shortness of breath" - Triggered emergency banner
- "Joint pain, fatigue, hand swelling" - Rheumatological condition info
- "Dizzy and nauseous for a week" - Differential analysis

Drug Interactions:

- "Ibuprofen and aspirin" - NSAID interaction risks
- "Warfarin and aspirin" - Severe bleeding risk flagged

- "Metformin and lisinopril" - Generally safe, monitoring noted
- "Sertraline and tramadol" - Serotonin syndrome risk identified

4. Edge Cases

- Empty query: Rejected with user-friendly message
- Out-of-domain query: Acknowledged limitations gracefully
- Rate limiting: Exponential backoff triggered correctly
- KB not built: Status indicator with rebuild prompt

Result: All 13 test cases passed. Edge cases handled gracefully.

Conclusion

MedAssist evolved from a concept into a complete, functional healthcare knowledge assistant demonstrating enterprise-grade AI architecture applied to medical information retrieval.

The RAG architecture proved highly effective for healthcare applications, eliminating hallucination risk by constraining the Llama 3.3 70B model to generate responses exclusively from retrieved FDA drug labels, PubMed abstracts, and MedlinePlus summaries.

The three specialized modules with tailored retrieval strategies demonstrate that a single knowledge base can serve diverse information needs through intelligent metadata filtering. The relevance scoring system provides users with an intuitive mechanism for assessing information reliability.

The adoption of FAISS over ChromaDB, PubMed over statistical APIs, and cosine-converted relevance scoring reflects informed technical decisions optimizing performance, data quality, and interpretability.

The development methodology using Claude Code for phase-by-phase construction proved highly effective for rapid delivery. The complete application was built and verified within a concentrated development session.

The project provides a foundation for future enhancements including additional medical databases, medical image analysis, multi-language support, and cloud deployment.

Limitations

1. Knowledge Base Scope

225 source documents covering 30 drugs, 15 disease categories, and 45 health topics. Queries about rare conditions or recently approved medications may not find relevant context.

2. Embedding Model Limitations

all-MiniLM-L6-v2 captures general semantic similarity but may miss domain-specific medical nuances. Specialized models like PubMedBERT could improve retrieval accuracy.

3. Static Authentication

Static credentials in session state. Production would require proper user management with encrypted storage and password hashing.

4. Single-Language Support

English only. Multi-language support would expand utility significantly.

5. No Real-Time Updates

Knowledge base is static until manually rebuilt. New drug approvals or research findings are not automatically incorporated.

6. No User Feedback

No mechanism for users to rate response quality or retrieval relevance for continuous improvement.

7. Limited Scalability

Single Streamlit server instance. Production would require load balancing and distributed vector storage.

8. FAISS Score Interpretation

Relevance percentages depend on the embedding model's training distribution and may not correspond precisely to human relevance judgments.

Glossary

Retrieval-Augmented Generation (RAG): An AI architecture that improves language model accuracy by retrieving relevant documents from a knowledge base before generating responses.

Vector Embedding: A numerical representation of text in high-dimensional space where semantically similar content occupies nearby positions.

FAISS: An open-source library by Meta for efficient similarity search over dense vectors.

Sentence Transformer: A neural network that converts text into fixed-dimensional vector embeddings capturing semantic meaning.

Cosine Similarity: A metric measuring angular similarity between two vectors, ranging from 0 to 1.

LangChain: An open-source framework for building applications with language models.

Groq LPU: Custom hardware by Groq for fast language model inference.

Prompt Template: A structured instruction format guiding the language model's generation behavior.

Chunk: A segment of text created by splitting larger documents for embedding and retrieval.

Metadata Filtering: Restricting similarity search to documents matching specific criteria.

Hallucination: The tendency of language models to generate plausible but incorrect information.

Session State: Streamlit's mechanism for persisting data across user interactions.